

Student: Samuele Mancini
Student id: 7190386
email: samuele.mancini@edu.unifi.it

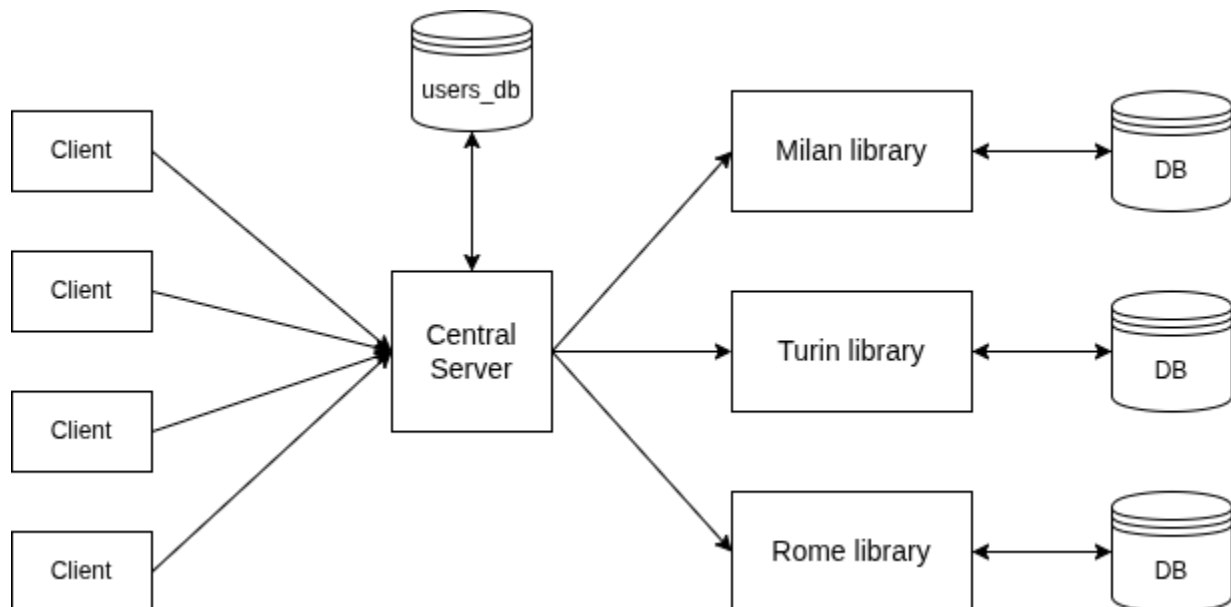
Report Library consortium

The project aims to develop a web application for a library consortium. Each library maintains its own book catalog and its database, while the central server is the only access point for the clients.

The system is composed by three actors:

- The user: through the web interface he can: register, login, search for a book, reserve a book and see its reservation
- The central server: manages the authentications, the users, the reservations, maintains a list of the libraries, forwards the search request to the libraries, merges the results from the libraries
- The library: manages its catalog, the book availability, responds to the central server's requests, updates the availability of the books

System architecture



The architecture is distributed: the book's information is stored only in the libraries while the central server manages the communication. The clients can communicate only with the central

server, the central server communicates with all the libraries with REST API. If a library is not available the system will continue to work properly.

Software architecture

The architecture of the application is organized in layers:
HTTP request -> Handler -> Service -> Repository -> Database.

Handler

The handlers manage the HTTP requests, then they read the parameters in the request, call the service and then they render the HTML template.

Service

They implement the business logic, for example: search for a book, merge the results from the libraries, manage the reservations, and communicate with the library service.

Repository

The repositories represent the access level to the data. It is very important because it allows testing very efficiently the interaction with the database.

Server communication

The communication between servers is based on HTTP requests. Each library exposes some REST API like GET /books?query=

The central server uses a HTTP client to make requests to the libraries and collect the results. Each request is managed in an independent way, allowing the server to ignore libraries temporarily not available.

Technologies Adopted

GORM: ORM library for Go

GORM is an object relational mapper that optimises the access with a database. It permits us to write queries as go code which then it translates in sql queries to interact with the database. In the project it was used to implement the repository interface both in the central server and in the libraries.

Gorilla

Gorilla is a library that completes the standard library, in particular it provides a collection of handler, cookies and file system session management. In this project it was used to route the request in the right handler and to manage the cookie session when a user logs in.

net/http: the standard package for managing HTTP and REST API.

html/template: used for rendering HTML pages, allowing the composition of HTML pages.

Use cases

Once a user browses for the application, a login web page is shown. If the user does not have an account he can create one in the register page.

Once logged in the user can search for a book by typing a title or an author (or part of it) in the search form. The browser will send an HTTP request to the central server, then an handler will call the search service. It will send requests to all the libraries with the data provided. Then the libraries will reply and the service will merge the response ignoring not available libraries. Finally the results will be sent to the template engine that will generate the HTML page that the user will visualize.

When a user wants to reserve a book it has to click on the button "Reserve" next to the book he searched and then a handler will call the reservation service. It will identify the library owning the book and make a request to it. The library will update the availability of the book and send a response. If it is ok the central server will add the reservation on its database and the browser page will refresh with the new data.

In a similar way when a user wants to delete a reservation it has to go on his profile, where all his reservation will appear, and click on the button of release next to the book.

Security considerations

The application implements basic security measures to protect users and their data.

The user's passwords are hashed before being saved in the database using the “bcrypt” library, in this way even if the database is compromised the user's password would be unreadable.

User authentication is managed through server side Gorilla sessions. After a successful login, the server creates a session identified by a cookie and protected pages verify the session before allowing access, blocking the reservation or the release of a book.

Moreover, all user inputs are validated before being processed to mitigate the risk of injection attacks.